



MÓDULO 5 · T15

Persistencia y Arquitectura

 *Organización y mappers de datos*

Carlos Rodríguez Contreras · GitHub: [karrocon](#)

¿Qué aprenderás hoy?

En esta sesión construirás una base sólida sobre cómo organizar el código en `src/` con capas claras, separar tipos y utilidades, e implementar el patrón mapper para transformar datos de la API al dominio de la aplicación.

1

Arquitectura `src/`

Organizar el código con capas claras: componentes, hooks, screens, types y utils.

2

Separación de tipos

Distinguir los tipos del dominio (español, limpios) de los tipos de la API externa.

3

Patrón mapper

Implementar `mapToPokemon` para transformar respuestas de API en tipos del dominio.

4

Separación de capas

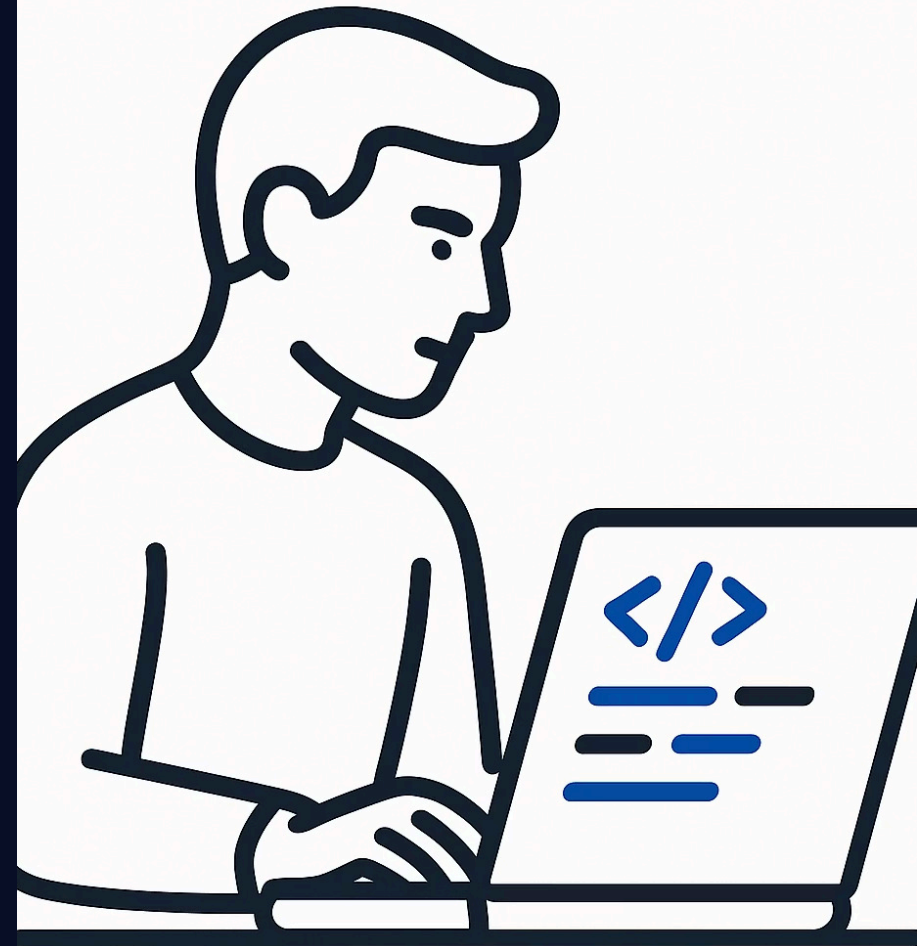
Entender por qué la UI no debe conocer la estructura interna de la API externa.

Estructura de carpetas recomendada

Una arquitectura clara en `src/` facilita el mantenimiento, la colaboración y las pruebas. Cada carpeta tiene una responsabilidad única: los tipos de la API nunca se mezclan con los del dominio, y el mapper actúa como frontera entre ambos mundos.

```
src/
├── components/ → UI reusable (PokemonTypeBadge, etc.)
├── constants/ → tipoColor, theme, navigation keys
├── hooks/ → usePokemons, useTheme, etc.
├── screens/ → PokedexScreen, SettingsScreen, etc.
├── types/
│   ├── pokemon.ts → tipos del dominio
│   └── pokeApi/ → tipos de la API externa
├── utils/
│   ├── mapToPokemon.ts → mapper API → dominio
│   └── storage.ts → AsyncStorage wrappers
```

❗ Mantener los tipos de la API en su propia carpeta `pokeApi/` deja claro que esos tipos son externos y no deben usarse directamente en la UI.



Tipos de dominio vs tipos de API

API (externo)

La PokéAPI devuelve los datos en inglés con una estructura anidada que refleja el diseño del servidor, no las necesidades de la app.

```
// PokéAPI devuelve en inglés
{
  name: 'charizard',
  types: [{ type: { name: 'fire' } }],
  stats: [{
    base_stat: 78,
    stat: { name: 'hp' }
  }]
}
```

Dominio (interno)

Nuestra app trabaja con tipos propios, en español, con la estructura exacta que la UI necesita – sin anidado innecesario.

```
// Nuestra app trabaja en español
{
  nombre: 'charizard',
  tipos: ['fuego'],
  stats: {
    hp: 78,
    ataque: 84,
    ...
  }
}
```

El patrón mapper

El mapper es una función pura ubicada en `utils/mapToPokemon.ts` que recibe los datos crudos de la API y devuelve un objeto del dominio perfectamente tipado. Toda la lógica de transformación queda en un único lugar.

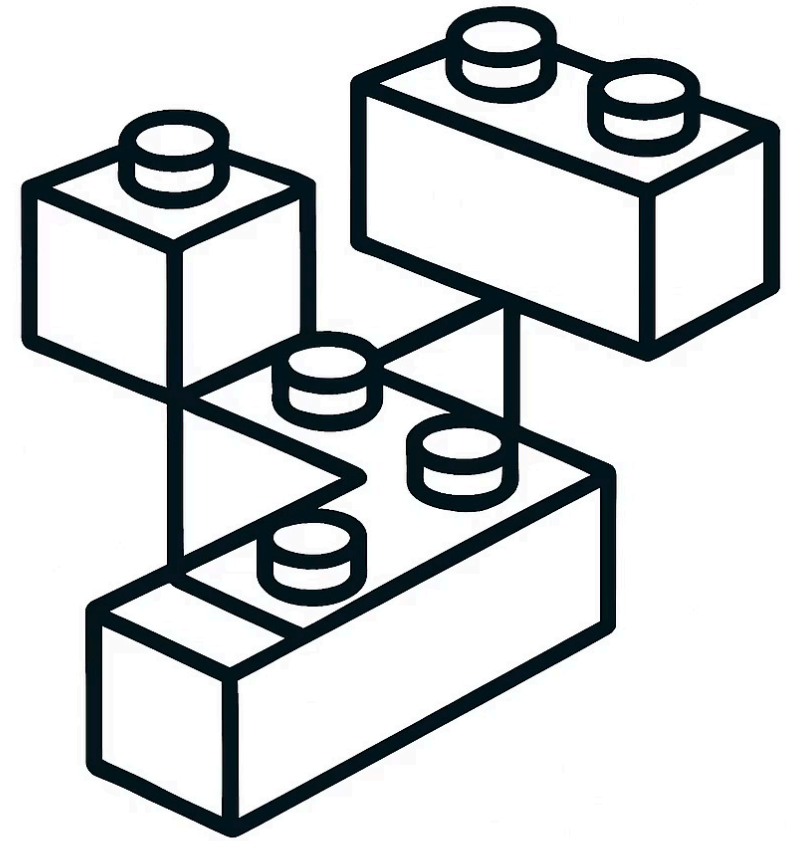
```
// utils/mapToPokemon.ts
export function mapToPokemon(
  r: GetPokemonResponse,
  species: GetPokemonSpeciesResponse
): Pokemon {
  return {
    id: r.id,
    nombre: r.name,
    tipos: r.types.map(mapToTipo), // inglés → español
    stats: mapToStats(r.stats),
    descripcion: mapToDescripcion(species),
    // ...
  };
}
```

- ❗ Al centralizar la transformación en el mapper, el resto del código trabaja siempre con tipos del dominio – nunca toca directamente la estructura de la API.

Mapper de tipos: inglés → español

La función `mapToTipo` usa un `Record<string, TipoPokemon>` como tabla de traducción. Si la API devuelve un tipo desconocido, se lanza un error explícito – esto evita fallos silenciosos difíciles de depurar.

```
function mapToTipo(
  t: { type: { name: string } }
): TipoPokemon {
  const map: Record<string, TipoPokemon> = {
    fire: 'fuego',
    water: 'agua',
    grass: 'planta',
    electric: 'eléctrico',
    fighting: 'lucha',
    // ... todos los tipos
  };
  const tipo = map[t.type.name];
  if (!tipo) throw new Error(
    `Tipo desconocido: ${t.type.name}`
  );
  return tipo;
}
```



Mapper de stats

La función `mapToStats` recorre el array de stats de la API y distribuye cada valor en el campo correspondiente del tipo `PokemonStats` del dominio. El `switch` hace la correspondencia explícita entre los nombres en inglés de la API y los campos en español del dominio.

```
function mapToStats(
  stats: GetPokemonResponse['stats']
): PokemonStats {
  const s: PokemonStats = {
    hp: 0, ataque: 0, defensa: 0, ...
  };
  stats.forEach(({ base_stat, stat }) => {
    switch (stat.name) {
      case 'hp':      s.hp = base_stat; break;
      case 'attack':  s.ataque = base_stat; break;
      case 'defense': s.defensa = base_stat; break;
      case 'special-attack': s.ataqueEspecial = base_stat; break;
      case 'special-defense': s.defensaEspecial = base_stat; break;
      case 'speed':   s.velocidad = base_stat; break;
    }
  });
  return s;
}
```

📌 **Buena práctica:** inicializar todos los campos a 0 antes del bucle garantiza que el objeto siempre tenga la forma completa de `PokemonStats`, aunque la API omita algún stat.

Ventajas del mapper

Aislamiento del cambio

Si la PokéAPI cambia su estructura de respuesta, solo actualizas el mapper. El resto del código, incluyendo la UI y los hooks, permanece intacto.

Código tipado en español

Todo el dominio de la aplicación usa nombres en español y tipos propios. Los desarrolladores trabajan con conceptos del negocio, no con la jerga de la API.

Testeabilidad inmediata

El mapper es una función pura: dado un objeto de respuesta de API, siempre devuelve el mismo objeto de dominio. No hay efectos secundarios, no hay red, no hay estado.

Separación de capas

La UI no conoce la estructura de la API. Si mañana cambias de PokéAPI a otra fuente, solo reescribes el mapper – nada más.

- ✓ Los tests del mapper son sencillos: construyes un objeto de respuesta de API, llamas al mapper y compruebas el resultado. Sin mocks complejos.

Tour starter/mapToPokemon.ts

Este es el fichero que encontrarás en la carpeta `starter/`. Contiene las firmas de las dos funciones marcadas como `TODO` que deberás implementar durante el ejercicio. Las funciones lanzan un error hasta que las completes – así sabrás si faltan implementaciones.

```
export function mapToPokemon(  
  response,  
  species  
): Pokemon {  
  throw new Error('TODO: implementar mapToPokemon');  
}  
  
export function mapToPokemons(  
  responses[],  
  speciesArr[]  
): Pokemon[] {  
  throw new Error('TODO: implementar mapToPokemons');  
}
```

⚠ No modifiques los tipos de la carpeta `types/pokeApi/` – esos tipos reflejan la respuesta real de la API y no deben alterarse.

Solución — solucion/mapToPokemon.ts

La solución completa define `TipoPokemon`, `PokemonStats` y `Pokemon`, e implementa los tres mappers encadenados. Compara tu implementación con esta referencia.

```
export function mapToPokemon(
  r: GetPokemonResponse,
  species: GetPokemonSpeciesResponse
): Pokemon {
  return {
    id: r.id,
    nombre: r.name,
    tipos: r.types.map(mapToTipo),
    stats: mapToStats(r.stats),
    descripcion: mapToDescripcion(species),
  };
}
```

```
export function mapToPokemons(
  responses: GetPokemonResponse[],
  speciesArr: GetPokemonSpeciesResponse[]
): Pokemon[] {
  return responses.map((r, i) =>
    mapToPokemon(r, speciesArr[i])
  );
}
```

✓ Si TypeScript no muestra ningún error y los Pokémon aparecen con nombres y tipos en español, ¡lo has conseguido! 🎉

Ejercicio T15

Pon en práctica todo lo aprendido. Abre el proyecto en tu editor y sigue los pasos en orden – cada uno construye sobre el anterior. Implementa los mappers de manera que TypeScript no muestre ningún error.

1 Abre el fichero de inicio

Navega a `starter/mapToPokemon.ts` en tu editor.

2 Define los tipos del dominio

Define `TipoPokemon`, `PokemonStats` y `Pokemon` en `src/types/pokemon.ts`.

3 Implementa `mapToTipo`

Crea el `Record<string, TipoPokemon>` con la traducción inglés → español para los 18 tipos.

4 Implementa `mapToStats`

Recorre el array de stats con el `switch` y rellena el objeto `PokemonStats`.

5 Implementa `mapToPokemon` completo

Ensambla todos los mappers parciales en la función principal `mapToPokemon`.

Resumen de la sesión

Estos son los conceptos clave que has trabajado hoy. Dominarlos te permitirá construir aplicaciones React Native con una arquitectura limpia, mantenible y completamente tipada con TypeScript.

Concepto	Elemento clave	Para qué sirve
src/types/	Tipos del dominio (español, limpios)	Contrato interno de la aplicación
src/types/pokeApi/	Tipos de la API externa (no cambiar)	Reflejo fiel de la respuesta del servidor
Mapper	<code>mapToPokemon(r, species)</code>	Transforma API → dominio en un único lugar
Función pura	Sin efectos secundarios	Fácil de testear, predecible y reutilizable
Record<K, V>	Diccionario tipado para el switch de tipos	Garantiza cobertura exhaustiva de todos los casos
Separación de capas	UI → hooks → mapper → API	La UI no conoce la estructura de la API

✅ ¡Enhorabuena! Ahora tienes las herramientas para estructurar proyectos React Native con capas bien separadas y mappers robustos. 🚀